CS 513 Final Project Proposal

Title

NutriMatch: A Hybrid Recipe Recommendation System with Dietary Constraints and Agentic Explanations

1. Project Summary

We propose to build a real-world **recommendation system** that suggests recipes to users based on their historical preferences, ingredient interests, dietary constraints, and practical needs such as preparation time. The project will compare multiple recommendation techniques and show how a lightweight hybrid design can improve recommendation quality without requiring expensive computation.

Our goal is to build a project that is:

- clearly aligned with the course requirement for a recommendation system
- strong on methodology comparison
- interesting and slightly beyond the syllabus
- feasible for a three-person team on normal laptops

2. Problem Statement

Recipe discovery is difficult because users often have multiple constraints at once: taste, nutrition, available ingredients, cooking time, and dietary restrictions. Generic search is not personalized, and pure popularity-based ranking often ignores health or preference differences.

The core question of this project is:

How can we build a practical recommendation system that gives personalized, explainable recipe suggestions while remaining accurate, interpretable, and computationally lightweight?

3. Why This Is a Strong Final Project

This topic is a good fit for an A-level project because it matches the project rubric closely:

- **Novelty**: recipe recommendation is more distinctive than common movie or credit-risk examples
- **Techniques used**: we can compare several methods rather than relying on one model
- **Comparison of results**: the project naturally supports side-by-side evaluation
- **Uniqueness of data source**: food, nutrition, and interaction data are richer and more original than many standard classroom datasets
- **Beyond the syllabus**: hybrid recommendation and an optional agentic assistant go beyond the core lecture topics
- **Presentation quality**: this project is easy to demo live because users can enter preferences and immediately see recommendations

4. Data Source Plan

Primary Dataset

We plan to use the public **Food.com Recipe & Review Data** dataset, with the complete download available via its linked **Kaggle page**.

This dataset includes raw recipe and interaction tables such as `RAW_recipes.csv` and `RAW_interactions.csv`, along with preprocessed interaction splits. It includes:

- user-recipe ratings or interactions
- recipe titles
- ingredient lists
- tags or categories
- preparation time
- nutrition-related metadata
- cooking steps or descriptions

This dataset is attractive because it supports both:

- **collaborative filtering**, using user-item interactions
- **content-based recommendation**, using ingredients, tags, and recipe metadata

Why This Dataset Works Well

- It is realistic and useful.
- It allows multiple modeling approaches.
- It supports interpretability.
- It does not require GPU training.
- We can reduce size if needed and still keep a meaningful project.
- It directly includes the metadata we need for a hybrid recommender: ratings, tags, ingredients, nutrition, and preparation time.

Backup Dataset

If access to the recipe dataset becomes inconvenient, our backups are:

- **MovieLens 1M** from GroupLens
- **Goodbooks-10k** from GitHub

This backup keeps the project structure intact and lowers project risk.

Data Availability Notes

- The primary Food.com dataset is the best thematic fit for our proposal because it contains both user-item interactions and rich item metadata.
- The BERD page is a stable public metadata record and points to the full Kaggle download.
- If Kaggle access becomes inconvenient, we can switch to MovieLens 1M immediately with almost no change to the modeling pipeline.

Dataset Risk Control

To keep the workload manageable, we will:

- remove missing or duplicate records
- keep users with a minimum number of ratings
- keep recipes with enough interaction history
- sample the dataset if it is too large
- use sparse matrices and CPU-friendly algorithms

5. Proposed Methodology

We will implement and compare the following approaches:

Baseline 1: Popularity-Based Recommendation

Recommend globally popular recipes. This gives us a simple baseline that is easy to interpret but not personalized.

Baseline 2: Content-Based Recommendation

Use recipe ingredients, tags, cuisine type, and metadata to build item profiles. Possible tools:

- TF-IDF on ingredient lists or tags
- cosine similarity
- optional nutrition/time filters

This method is especially useful for **cold-start recipes** or users with little interaction history.

Model 3: Collaborative Filtering

Use user-item interactions to learn taste similarity. Possible methods:

- user-based or item-based kNN
- matrix factorization such as SVD if time allows

This is likely to outperform the popularity baseline when enough interaction data exists.

Model 4: Hybrid Recommendation System

Combine collaborative and content-based methods using a weighted score or fallback strategy. This is our intended final system because it balances:

- personalization
- interpretability
- cold-start handling
- manageable computation cost

Stretch Extension: Agentic Recommendation Assistant

For extra credit and differentiation, we may add a lightweight **agentic layer** that:

- converts natural-language preferences into structured filters
- explains why a recipe was recommended
- suggests substitutions for dietary restrictions

Example input:

> "Recommend quick high-protein dinners with no peanuts and no pork."

This extension would not train a large model. Instead, it would use an LLM only for preference parsing and explanation generation, keeping computation low while adding originality.

6. Evaluation Plan

Because the course emphasizes comparison of techniques, evaluation will be a major part of the project.

Offline Evaluation Metrics

For recommendation quality, we plan to use:

- Precision@K
- Recall@K
- NDCG@K
- coverage
- possibly diversity or novelty if time allows

If we use explicit rating prediction as a secondary analysis, we may also report:

- RMSE
- MAE

Comparison Questions

We want to answer the following:

- Does collaborative filtering beat the popularity baseline?
- Does the hybrid system outperform single-method models?
- Does content-based recommendation help in cold-start situations?
- Do nutrition/time filters improve usefulness for real users?

Qualitative Evaluation

We will also show sample recommendation cases to demonstrate:

- personalized results
- interpretable explanations
- practical usability

7. Software and Tools

Planned tools:

- Python
- pandas
- numpy
- scikit-learn
- Surprise or another lightweight recommendation library

- matplotlib / seaborn
- Streamlit or notebook-based demo
- optional LLM API for the agentic explanation layer

All of these are feasible on standard laptops. No heavy deep learning training is required.

8. Deliverables

Our final submission will include:

- cleaned and documented dataset pipeline
- exploratory data analysis
- baseline and advanced recommendation models
- model comparison tables and figures
- final hybrid recommender
- short live demo or interactive notebook
- final presentation slides
- final report with references and documentation

9. Team Work Division

To keep the workload balanced for a three-person group, we propose the following split:

Member A: Data and Content Pipeline

- dataset acquisition and cleaning
- EDA and visualization
- content-based recommender
- documentation of data preprocessing

Member B: Collaborative Filtering and Evaluation

- user-item matrix construction
- kNN / SVD collaborative model
- train/test splitting
- evaluation metrics and comparison tables

Member C: Hybrid System, Demo, and Presentation

- hybrid ranking logic
- dietary/time filtering interface
- optional agentic explanation layer
- demo preparation and slide design

Shared Responsibilities

- problem framing
- interpretation of results
- final presentation
- final paper polishing

10. Workload and Compute Control

This project is intentionally designed to be ambitious in presentation quality, but moderate in implementation cost.

We will control workload by:

- avoiding large neural network training
- using CPU-friendly methods first
- building the project in layers: baseline -> content -> collaborative -> hybrid
- keeping the agentic feature optional rather than making it a dependency
- using a reduced dataset sample if runtime becomes an issue

This makes the project realistic for a three-person team while still looking advanced.

11. Proposed Timeline

Week 1

- finalize dataset
- clean data
- perform EDA
- implement popularity baseline

Week 2

- build content-based recommender
- build collaborative filtering model
- define evaluation pipeline

Week 3

- build hybrid recommender
- run comparisons
- create charts and result tables

Week 4

- add optional agentic explanation feature
- prepare demo
- finalize slides and paper

12. Risks and Mitigation

Risk 1: Dataset is too large or noisy

Mitigation:

- sample the dataset
- filter sparse users/items
- keep a backup dataset ready

Risk 2: Collaborative filtering is weak because of sparsity

Mitigation:

- rely on content-based features
- use a hybrid fallback strategy

Risk 3: LLM or API access is limited

Mitigation:

- keep the agentic component optional
- replace it with rule-based preference parsing if necessary

Risk 4: Team workload becomes uneven

Mitigation:

- assign clear ownership early
- keep shared evaluation and presentation checkpoints

13. Expected Outcome

We expect the hybrid system to outperform the popularity baseline and provide more practical recommendations than any single method alone. We also expect the project to stand out because it combines:

- a real-world recommendation problem
- multiple methods and direct comparison
- an interpretable and demo-friendly interface
- a lightweight extension beyond the syllabus

14. Conclusion

This project is ambitious enough to compete for a top grade, but controlled enough to finish well with three people. It directly satisfies the course project requirement, supports strong comparisons, uses an interesting real-world dataset, and includes a realistic path for an extra-credit agentic extension without high compute cost.

If approved, this will give us a project that is both **practical** and **presentation-friendly**, which is exactly the kind of profile that should perform well in CS 513.